Information Security
Information Security is the protection of information and its critical elements. This includes the systems, hardware, software, and communication channels that use, store, process, and transmit information.
The main goal is to ensure that information remains safe, reliable, and available to authorized users while preventing unauthorized access or damage.
Information security generally focuses on protecting:

- **Data** (files, messages, transactions, records)
- **Devices** (servers, computers, phones)
- **Networks** (internet, Wi-Fi, internal networks)
- **Applications and services** that handle data

Key Security Terms
Vulnerability
A vulnerability is a weakness in a system, network, software, or process that can be exploited by an attacker.
Examples of vulnerabilities:

- Weak passwords
- Unpatched software
- Misconfigured servers
- Lack of access controls

Threat
A threat is a possible danger that may exploit a vulnerability.
Threats can come from:

- Hackers (external attackers)
- Insider misuse (employees, staff)
- Malware (viruses, ransomware)
- Natural disasters (floods, fires)

Simple relation:
Vulnerability = weakness
Threat = danger that can use that weakness

Security Services Categories
Information security is commonly described using these **six security services**:

1. **Confidentiality**
2. **Authentication**
3. **Access Control**
4. **Integrity**
5. **Non-Repudiation**
6. **Availability**

These services help protect data and systems from different types of attacks.
Confidentiality
Confidentiality means protecting information from being disclosed to unauthorized users. It mainly defends against passive attacks, where attackers try to “observe” data without changing it.

Purpose
The core purpose is Ensure information is accessible only to authorized people/systems.

Confidentiality and Traffic Analysis
Modern confidentiality also includes protection against traffic analysis.
Traffic Analysis is when attackers do not read the message content, but they analyze:

- Who is communicating with whom
- The frequency of messages
- Message lengths
- Communication timing patterns

Even if data is encrypted, traffic patterns can reveal sensitive information.
Example

- During an online payment, the credit card number and payment details must remain confidential.

Authentication
Authentication ensures that communication is genuine and that the sender/receiver is truly who they claim to be.

Authentication depends on the scenario:

A) Authentication of a Single Message
Ensures the receiver can confirm that:

- The message really came from the original sender
- The message was not forged

B) Authentication of an Ongoing Interaction
Ensures that during a continuous connection:

- Both entities are legitimate
- No third party is interfering
- A third party cannot masquerade as one side

Example

- Logging into an email account using password + OTP
- Secure communication between a user and a bank system

Access Control
Access Control is the ability to restrict who can use:

- Systems
- Resources
- Applications
- Data (read, write, modify, delete)

Key idea
To apply access control properly:

1. The entity must first be identified (Authentication)
2. Then permissions are assigned (Authorization)

Access control ensures users only get the access they are allowed to have.

Example

- A student can view results, but cannot change them
- Only HR can access salary records

Integrity

Integrity means ensuring that information remains accurate and unchanged, unless changed by authorized actions.

Integrity applies to:

- A single message
- A stream of messages (connection-oriented)

A) Message Integrity (Single Message)

Protects against:

- Modification of the message content

B) Connection-Oriented Integrity (Stream of Messages)

Assures that messages are received exactly as sent, with protection against:

- Duplication
- Insertion
- Modification
- Reordering
- Replay attacks (sending old messages again)

Example

- In online banking, the amount "$100" must not be changed to "$1000" during transmission.

Non-Repudiation

Non-repudiation prevents the sender or receiver from denying that a message or transaction took place.

It provides proof for later verification.

Two perspectives:

A) Sender cannot deny sending

The receiver can prove later that:

- The genuine sender actually sent the message

B) Receiver cannot deny receiving

The sender can prove later that:

- The genuine receiver actually received it

Example

- Digital signatures in legal documents
- Email confirmation receipts in critical systems

Availability
Availability ensures that systems and information are:

- Accessible
- Usable
- Available on demand
  for authorized users according to performance requirements.

Availability can be heavily affected by Denial of Service (DoS) attacks.
Example

- A website goes down because attackers overload the server with traffic
- A hospital system becomes unavailable during an emergency

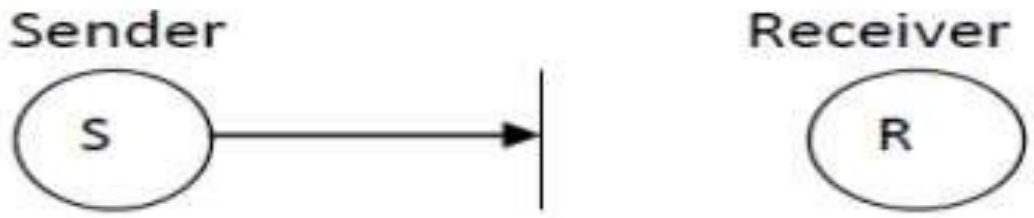
Security Attacks by Effect
Security attacks are often classified by their **impact/effect** on information and systems:

1. **Interruption**
2. **Interception**
3. **Modification**
4. **Fabrication**
5. **Repudiation**
6. **Unauthorized Access**

Interruption:
Interruption occurs when an asset becomes:

- Destroyed
- Unavailable
- Unusable

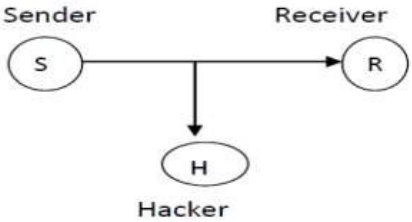
This means authorized users cannot access or use the information/services.



Examples

- Jamming wireless signals
- Disabling file management systems
- Server crash caused intentionally

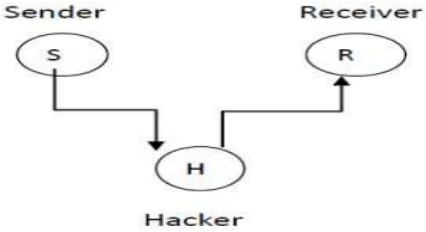
Interception:
Interception happens when an unauthorized party gains access to information.



Examples

- Eavesdropping on a network
- Packet sniffing
- Reading someone's private emails without permission

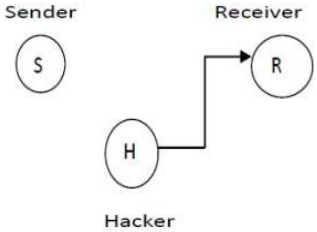
Modification
Modification means an attacker changes data during storage or transmission.



Examples

- Changing grades in a university system
- Altering bank transaction amounts
- Modifying a message content before it reaches the receiver

Fabrication:
Fabrication means creating fake data or messages and inserting them into the system.



Examples

- Fake emails (spoofing)
- Fake bank transaction messages
- Creating false logs or records

Repudiation
Repudiation occurs when someone denies:

- Sending a message
- Receiving a message
- Performing an action

Example

- A user denies making an online purchase after receiving the product

Unauthorized Access (Attack on Access Control)
Unauthorized access happens when someone accesses resources without permission.

Examples

- Using stolen passwords
- Accessing admin panels without authorization
- Entering restricted systems

Main Security Mechanisms
Security mechanisms are the practical tools and techniques used to prevent, detect, and respond to threats. They work together to protect information and systems from unauthorized access, misuse, damage, or disruption.

1) Information Flow Control
Information flow control manages and restricts how information moves between different parts of a system or network. The goal is to ensure that sensitive information does not flow into areas where it should not be accessible.
It is especially important in:

- Organizations with multiple departments (HR, Finance, IT)
- Systems with different security levels (public vs confidential zones)

Example:
A company may allow employees to access general files but restrict financial documents from moving to public network segments.

Access Control Matrix (ACM)
An Access Control Matrix (ACM) is a structured method to define who can access what.
It exists in:

- Operating Systems
- Databases

It describes the allowed access rights by mapping:

- **Subjects** (users, processes)
- **Objects** (files, tables, applications)
- **Permissions** (read, write, execute, delete)

Example:
A student may have “read” access to results, while an admin has “read + write” access.

Firewalls
A Firewall is a security mechanism that monitors and controls traffic between networks. Most commonly, it controls communication between:

- The Internet
- The local system / internal network (intranet)

Firewalls enforce rules such as:

- Allowing safe traffic (e.g., browsing)
- Blocking suspicious traffic (e.g., unauthorized ports, malicious IPs)

Example:
Blocking incoming connections to a database server from the internet to prevent external attacks.

Cryptography
Cryptography is the process of converting information between:

- Plaintext (readable form)
- Ciphertext (unreadable encrypted form)

The process works like this:

- Sender converts plaintext → ciphertext (encryption)
- Receiver converts ciphertext → plaintext (decryption)

Cryptography helps protect:

- Confidentiality (privacy)
- Integrity (prevent tampering)
- Authentication (prove identity, in some cases)

Example:
During online banking, encryption keeps account details safe while transferring data over the internet.

Digital Signatures
A digital signature is used to ensure:

- Authenticity (who sent it)
- Integrity (not modified)
- Non-repudiation (sender cannot deny)

Digital signatures are commonly used for:

- Documents
- Software updates
- Emails and messages

Example:
A digitally signed software update confirms it is from the real company and was not modified by attackers.

Intrusion Detection Systems (IDS)
Intrusion detection is used to detect:

- Attempted attacks
- Successful attacks
- Suspicious activities inside a system or network

IDS can monitor:

- Network traffic patterns
- Login attempts
- System behaviour and anomalies

Example:
If someone tries many wrong passwords quickly, IDS can detect it as a brute-force attack.

Auditing
Auditing is maintaining a record (log) of all security-related events in a system.
It provides accountability and helps in investigations.
Auditing can track:

- Logins and logouts
- Failed login attempts
- Access to sensitive files
- Changes in system settings
- Administrative actions

Example:
If a database record is changed, audit logs can show who changed it and when.

Steps to Handle Security Risks
Organizations handle security risks through a structured approach that ensures threats are managed throughout the lifecycle of an incident.

1) Identify
The first step is to understand:

- What resources exist (servers, apps, data)
- What risks and vulnerabilities are present
- Which assets are most important and sensitive

This step helps organizations know:

- What needs protection
- What threats are most likely

Example:
Identifying that customer personal data is the most critical asset and must be protected strongly.

2) Protect
In this step, security protection mechanisms are deployed to reduce risk.
Protection includes:

- Strong passwords and authentication
- Access control policies
- Encryption

- Firewalls
- Regular updates and patching

Example:
Applying encryption and restricting database access to only authorized staff.

3) Detect
If an attack occurs, the organization must detect it quickly.
Detection mechanisms include:

- Intrusion detection systems (IDS)
- Monitoring tools
- Alerts for suspicious activities
- Log analysis

Example:
Getting alerts when unusual traffic spikes occur or when unauthorized login attempts happen.

4) Respond
When a breach is detected, response is necessary to:

- Contain the threat
- Stop further damage
- Reduce impact

Response actions include:

- Blocking attacker IPs
- Isolating infected devices
- Disabling compromised accounts
- Investigating the breach

Example:
If malware is found, isolating the affected system from the network to prevent spread.

5) Recover
After the incident, recovery focuses on restoring normal operations.
Recovery includes:

- Restoring services and data backups
- Fixing vulnerabilities
- Updating security controls
- Improving policies to prevent future incidents

Example:
Restoring data from backups and applying security patches to prevent the same breach again.

Information Security Risk

Introduction

The primary mission of information security is to ensure that an organization's systems and data remain protected and reliable. In simple words, it ensures that systems and their contents stay correct, safe, and usable.

If threats did not exist, organizations could spend most of their time improving systems for better performance and user experience. However, the reality is different: attacks on information systems happen daily, and organizations must continuously manage risks to protect operations, customers, and business reputation.

Business Needs First

Information security is not only a technical requirement—it is a business requirement. It supports an organization by performing four key functions:

1. **Protecting the organization's ability to function**
2. **Enabling the safe operation of applications**
3. **Protecting the data the organization collects and uses**
4. **Safeguarding technology assets**

These functions highlight that information security should always be aligned with business objectives, cost, and impact.

Protecting the Ability of an Organization to Function

Information security is the responsibility of both:

- **General management**
- **IT management**

It is not only a technology issue—information security is also:

- A **management issue**
- A **people issue**

Organizations must address information security by evaluating:

- How security incidents impact business operations
- What the cost of prevention is compared to the cost of a breach

A security failure can disrupt critical processes such as payments, customer support, supply chain, and internal communication, which directly affects the organization's ability to function.

Enabling the Safe Operation of Applications

Organizations rely on applications to run business functions, such as:

- Payroll and HR systems

- Customer databases (CRM)
- E-commerce systems
- Clinical, banking, or government platforms

To keep these applications safe, organizations must build secure IT environments and maintain them continuously. Management must not install systems and then shift all responsibility to the IT department. Instead, management must continue oversight to ensure:

- Proper policies are followed
- Controls remain active and updated
- Security is aligned with business needs

Protecting Data that Organizations Collect and Use

Data is one of the most valuable assets in modern organizations. Without data, an organization may lose:

- Records of transactions
- Business history and accountability
- Customer trust
- Ability to deliver value to customers

This is why protecting:

- **Data in motion** (data being transmitted across networks)
- **Data at rest** (data stored in databases, servers, devices)

are both critical in information security. Security risks can target either one, so organizations must protect both.

Safeguarding Technology Assets in Organizations

Organizations must secure their technology resources such as:

- Networks
- Servers
- End-user devices
- Infrastructure services
- Cloud services (if used)

As an organization grows, security needs increase. The company may require:

- Additional security services
- More advanced controls

- Stronger tools to replace old systems

Security programs that worked for a small organization may become inadequate as operations expand, meaning more robust security solutions become necessary over time.

Threats

Definition of Threat

A threat is any object, person, or entity that represents a constant danger to an organization's asset.
Management must understand the different threats facing the organization so they can plan resources and controls properly.

Major Types of Threats

Deliberate Software Attacks (Malware)

Deliberate software attacks involve malicious programs designed to:

- Damage systems
- Destroy data
- Deny services (make systems unavailable)

These attacks are commonly called **malware**, and they include:

- **Viruses**: attach to files/programs and spread when executed
- **Worms**: spread automatically through networks without user action
- **Trojan Horses**: look legitimate but carry malicious behavior
- **Logic Bombs**: activate when specific conditions occur (time/event-based)
- **Backdoor/Trapdoor**: hidden access path bypassing normal authentication
- **Polymorphic threats**: change their code to avoid detection
- **Virus/Worm hoaxes**: false warnings that cause confusion and disruption

Malware can cause operational failure, financial loss, and data leakage.

Deviations in Quality of Service

Quality of service threats occur when services are not delivered as expected. Since modern organizations depend on multiple support systems, a failure in one can impact the entire information system.

Key causes include:

Internet Service Issues

- Internet Service Provider (ISP) failures can significantly reduce availability
- Organizations using outsourced hosting depend on providers for:

    - Internet service
    - Hardware
    - Operating systems
    - Website uptime and maintenance

Any provider disruption can undermine business operations.

Power Irregularities

Power issues are very common and can heavily affect systems. Organizations without properly conditioned power become vulnerable.

Common power problems include:

- **Fluctuations**: short or long variations
- **Excesses (spikes/surges)**: sudden voltage increase
- **Shortages (sags/brownouts)**: low voltage levels
- **Losses (faults/blackouts)**: complete power loss

Organizations apply controls such as UPS systems, generators, and power conditioners to reduce damage.

Espionage or Trespass

Espionage or trespass refers to unauthorized access to protected information.

It includes:

- **Competitive intelligence (legal)**: collecting public or legal market info
- **Industrial espionage (illegal)**: stealing private company information

Common methods include:

- **Shoulder surfing**: watching someone enter passwords or view confidential data
- Hackers bypass controls using skill, fraud, or deception

Security controls should also communicate boundaries—so intruders know they are entering restricted cyberspace.

Types of Hackers

Expert Hacker

- Creates exploit scripts and attack tools
- Highly skilled in multiple areas
- Often shares tools with others

Unskilled Hacker

- Uses tools created by experts

- Larger in number than expert hackers
- Often does not fully understand systems they attack

Other Rule Breakers

Cracker

- Breaks software protections to enable illegal duplication or unauthorized access

Forces of Nature

Forces of nature are among the most dangerous threats because they can disrupt:

- Storage of information
- Transmission of information
- Use of information systems

Examples include:

- Floods
- Fires
- Earthquakes
- Storms

Organizations must implement controls and contingency plans such as:

- Disaster recovery strategies
- Offsite backups
- Business continuity planning

Human Error or Failure

Human error includes acts performed without malicious intent, yet they can still cause major damage.

Common causes:

- Inexperience
- Improper training
- Incorrect assumptions

Employees are often considered among the greatest threats to data because mistakes happen frequently in daily operations—such as sending sensitive files to the wrong person or misconfiguring access rights.

Information Extortion

Information extortion occurs when an attacker steals information and demands payment to:

- Return the information, or
- Prevent disclosure

This often happens in cases like:

- Credit card number theft
- Data breach ransom demands

Missing, Inadequate, or Incomplete Policies

If an organization has weak or incomplete security policy and planning, it becomes highly vulnerable to:

- Loss of assets
- Data disclosure
- Damage to systems

Without proper policies and controls, the organization is more likely to suffer greater impact when other threats lead to real attacks.

Sabotage or Vandalism

Sabotage or vandalism threats range from small damage to organized attacks.

Examples:

- Website defacement
- Destruction of data or services
- Disrupting organizational systems

These attacks can reduce consumer trust and cause financial damage.
Hacktivist and cyber activist activities are increasing in many cases.

Cyberterrorism is a more extreme and dangerous form, aiming to cause widespread fear or disruption.

Theft

Theft is the illegal taking of another's:

- Physical property (hardware devices)
- Electronic property (data)
- Intellectual property (trade secrets, designs, patents)

Physical theft is often easier to control with security guards and locks.
Electronic theft is more complex because it may not leave obvious physical evidence.

Technical Hardware Failures or Errors

Technical hardware failures occur when manufacturers distribute devices or equipment that contain design flaws or defects. These flaws can cause the system to

behave outside expected performance standards, which may lead to unreliable service, unexpected crashes, or incorrect results.

Hardware failures may be:

- **Terminal (permanent)**: the component fails completely and must be replaced.
- **Intermittent (temporary)**: the failure happens occasionally, making it harder to detect and troubleshoot.

Impact on security:
Hardware failures can reduce availability, damage stored data, and create opportunities for attackers if systems become unstable or misconfigured due to repeated failures.

Technical Software Failures or Errors

Software failures happen when purchased or deployed software contains hidden faults that were not revealed during testing. Sometimes, the interaction between certain hardware and software can expose new bugs, even if the software normally works fine in other environments.

Common causes:

- Incomplete testing
- Compatibility issues (hardware + OS + software)
- Poor coding practices
- Unpatched vulnerabilities

Because software bugs are frequent, there are many resources and communities dedicated to identifying and tracking these issues, including entire websites that document vulnerabilities and patches.

Impact on security:
Software failures can lead to data leakage, unauthorized access, system downtime, or vulnerabilities that attackers can exploit.

Technological Obsolescence

Technological obsolescence occurs when an organization continues using outdated infrastructure that no longer meets modern security and performance requirements. Old systems often become unreliable and untrustworthy because:

- They may not support new security updates
- Vendors stop providing patches
- Compatibility becomes weak with modern tools
- Security mechanisms may be outdated

Proper managerial planning and regular upgrades help prevent obsolescence. The IT department plays a major role in:

- Assessing system lifecycle
- Planning upgrades and replacements
- Ensuring security updates remain available

Impact on security:
Obsolete systems increase risk because attackers often target old, unpatched technologies.

Attacks

Definition of an Attack

An attack is an intentional act that exploits a vulnerability (a known weakness) in a controlled system. Attacks are typically carried out by a threat agent (attacker) with the goal to:

- Damage systems
- Steal information
- Disrupt services
- Gain unauthorized access

Common Types of Attacks

a) Malicious Code

This includes running harmful programs such as:

- Viruses
- Worms
- Trojan horses
- Active web scripts
  These attacks are used to destroy data, steal information, or control systems.

b) Hoaxes

A hoax is a deceptive message that may contain:

- A fake virus warning, sometimes attached with a **real virus**
  This causes panic and spreads malicious attachments quickly.

c) Back Door

A back door is a hidden access method used to bypass normal security controls. Attackers use:

- Known backdoors
- Newly discovered vulnerabilities
- Misconfigurations

d) Password Cracking

Password cracking attempts to obtain passwords through guessing methods:

- **Brute force:** tries every possible password combination
- **Dictionary attack:** uses a list of common passwords (the “dictionary”)
- **Targeted dictionary:** focuses on specific user accounts and likely passwords

e) Denial of Service (DoS)

In a DoS attack, the attacker sends a huge number of requests to a system. The target cannot handle both:

- Attack traffic, and
- Normal user traffic

This can cause:

- System slowdown
- System crash
- Service failure

f) Distributed Denial of Service (DDoS)

A DDoS is more powerful than DoS because it is launched from many locations at the same time, often using botnets.

g) Spoofing

Spoofing is a technique where an attacker pretends to be a trusted source, often by using a fake:

- IP address
- Email identity
- Website identity

h) Man-in-the-Middle (MITM)

In this attack, the attacker:

- Monitors network communication
- Modifies messages
- Inserts altered data back into the network
  This can lead to stolen credentials or altered transactions.

i) Spam and Mail Bombing

- **Spam:** unsolicited emails; often a nuisance but can carry malicious links or attachments
- **Mail bombing:** sending massive emails to overload a mailbox/server (also a DoS form)

j) Sniffers

A sniffer is a tool/device that monitors data traveling over a network. It can be used:

- Legitimately by admins for troubleshooting
- Illegally to steal usernames, passwords, or confidential data

k) Phishing

Phishing is an attempt to steal personal/financial information by pretending to be a legitimate organization (e.g., bank, company).

l) Pharming

Pharming redirects users from a real website to a fake website (even if the user types the correct web address). The goal is to steal sensitive data.

m) Social Engineering

Social engineering uses human manipulation to get confidential information (passwords, access details). It targets people instead of systems.

A key concept is that humans can become the weakest link if they are not trained properly.

n) Timing Attack

A timing attack may exploit system timing behaviour, such as exploring browser cache and cookie behaviour, to gain malicious advantages.

Secure Software Development

Why Secure Software Development Matters

Many information security problems are caused by weaknesses in software. Since most modern systems rely heavily on software, security must be built during development—not added later.

Organizations often use the Systems Development Life Cycle (SDLC) for building systems. Many have added security objectives into SDLC to produce secure software.

This approach is often called Software Assurance (SA) — it means building software in a way that ensures it is:

- Secure
- Reliable
- Compliant with security requirements

Software Design Principles (Security Principles)

Common Secure Design Principles

Good software development follows principles that reduce vulnerabilities and improve security:

1. **Keep the design simple and small**
   Smaller systems are easier to secure and test.
2. **Access decisions by permission, not exclusion**
   Default should be "deny" unless explicitly allowed.
3. **Every access to every object must be checked**
   Authorization should be verified each time.
4. **Security depends on possession of keys/passwords**
   Access must rely on strong authentication.
5. **Two-key protection for sensitive mechanisms**
   Some actions require two conditions or approvals.
6. **Least privilege**
   Users/programs should only have the minimum permissions needed.
7. **Minimize shared mechanisms**
   Reduce components shared across many users to limit widespread impact.
8. **Usable security**
   Interfaces must be easy so users naturally follow secure behaviour.

Software Development Security Problems

Common Problem Areas in Software Development

Many vulnerabilities happen due to poor coding, weak validation, or misconfigurations. Common issues include:

- **Buffer overruns (buffer overflow)**
- **Command injection**
- **Cross-site scripting (XSS)**
- **Failure to handle errors properly**
- **Failure to protect network traffic**
- **Failure to store/protect data securely**
- **Weak random number generation (not cryptographically strong)**

Other major security problems:

- Format string vulnerabilities
- Neglecting change control
- Improper file access
- Improper use of SSL/TLS
- Information leakage
- Integer overflow/underflow bugs
- Race conditions
- SQL injection

Additional weak practices:

- Trusting network address resolution
- Unauthenticated key exchange
- Using magic URLs or hidden forms as “security”
- Weak password-based systems
- Poor usability that leads to unsafe user behaviour

The Security Problem in Computing

Meaning of Computer Security

The meaning of computer security has changed over time. In earlier days, people mainly focused on physical protection of computers. Traditionally, computer facilities were protected to:

- Prevent theft or damage to hardware
- Prevent theft or damage to information
- Prevent disruption of service

Today, computer security covers protection of:

- Computing devices (computers, smartphones)
- Networks (private/public networks, including the Internet)
- Digital information and services

Computer security includes:

- **Physical security** (locks, restricted rooms, device protection)
- **Information security** (protecting data from unauthorized access, change, or destruction)

It is also commonly called Cyber Security or IT Security (though those terms often focus more on the digital side than physical security).

Important Terms in Computer Security

Vulnerability

A vulnerability is a weakness in a system that allows an attacker to reduce security.
It usually involves:

- A system flaw/susceptibility
- Attacker access to the flaw
- Attacker capability to exploit it

The collection of exploitable points is often called the attack surface. Vulnerability management is the continuous process of identifying, classifying, fixing, and reducing vulnerabilities (mostly software-related).

Backdoor

A backdoor is a method to bypass normal authentication and gain hidden access to a system. It may be:

- A separate installed program
- A modification inside existing software or hardware
- A hidden access path that stays undetected

Backdoors can also fake system information such as disk/memory usage.

Denial-of-Service (DoS) and DDoS

A DoS attack aims to make a system unusable (not to steal access). It can:

- Lock a user account by repeated wrong passwords
- Overload a server/network with requests

A DDoS attack uses many compromised machines ("zombies" in a botnet) to flood the target at once, causing resource exhaustion.

Direct-Access Attacks

If an attacker gets **physical access** to a device, they can:

- Install keyloggers or covert devices
- Copy large amounts of data to portable storage
- Boot from external media and read the hard drive

The strongest defence is encrypting storage and keeping the key separate. Direct-access attacks are a major threat even for standalone computers that never connect to the internet.

Eavesdropping

Eavesdropping is secretly listening to communications between hosts (often over a network).

Spoofing

Spoofing is when a person/program pretends to be another trusted identity by falsifying data to gain an advantage.

Tampering

Tampering is the intentional modification of a product/system to make it harmful or unsafe.

Repudiation

Repudiation happens when a party denies an action, such as challenging the authenticity of a signature or denying that they performed a transaction.

Information Disclosure

Information disclosure (privacy breach/data leak) is when information that should be secure is released into an untrusted environment.

Elevation of Privilege

Elevation of privilege is when a user/program gains permissions higher than what is normally allowed (e.g., normal user becomes admin).

Exploits

An **exploit** is code/data/commands that take advantage of a bug to cause unintended behaviour—often leading to:

- System control
- Privilege escalation
- Denial of service

Exploit code is often reused in Trojans and viruses.

Indirect Attacks

An **indirect attack** is launched using a third-party computer to hide the real attacker and make tracking difficult (e.g., anonymizing networks).

Computer Crime

Computer crime refers to any crime involving a computer and a network.

Cyber Crime Prevention Tips

1. Use **strong passwords** (mix letters, numbers, symbols; minimum length; change regularly)
2. Secure your computer:
    - Enable **firewall**
    - Install **anti-virus/malware**
    - Block spyware with **anti-spyware**
3. Be careful on social media (privacy settings + avoid oversharing)
4. Secure mobile devices (trusted apps only)
5. Install latest OS/app updates (use auto updates)
6. Protect your data (encryption + backups stored separately)
7. Secure Wi-Fi (change defaults; avoid sensitive actions on public Wi-Fi)
8. Protect e-identity (only share personal data on secure/trusted sites)
9. Avoid scams (don't click unknown links; verify sources)
10. Report incidents to the right authority/support when needed

Security Principles

Your notes list the main principles as:

- **Confidentiality**: only authorized users can view data
- **Integrity**: data must not be changed illegally
- **Authentication**: verify identity
- **Non-repudiation**: sender/receiver cannot deny actions
- **Access control**: who can access what
- **Availability**: systems/data available to authorized users at the right time

Attacks: Types

Security systems aim to ensure:

- No unauthorized disclosure
- No illegal modification
- Legitimate users can access data

Attacks are classified as:

- **Passive attacks**: observe only, no modification
- **Active attacks**: modify the message/system in some way

Elementary Cryptography

Core Concepts

- **Encryption**: converting readable text (**plaintext**) into unreadable form (**ciphertext**)
- **Decryption**: converting ciphertext back into plaintext
- A complete system is called a **cryptosystem**

Formal notation:

- Ciphertext: **C = E(P)**
- Plaintext: **P = D(C)**
- With a key: **C = E(K, P)**

Cryptanalyst

A **cryptanalyst** studies encrypted messages and tries to break encryption to reveal hidden meaning.

Confusion and Diffusion

- **Confusion**: ciphertext gives no clue about the original message
- **Diffusion**: spreads plaintext structure across ciphertext so patterns are harder to detect

Substitution Ciphers

Monoalphabetic Substitution (1:1 mapping)

Additive Cipher

Key value added to plaintext character values (0–25).
Example concept: **C = (P + K) mod 26**

Affine Cipher

Combination of multiplicative + additive:

- **C = [(P × K1) + K2] mod 26**
- Decryption uses the inverse of K1

Polyalphabetic Substitution

Each occurrence of a character can encrypt differently. Examples include:

- Auto-key cipher
- Playfair cipher
- Vigenère cipher
- Hill cipher

Transposition Cipher

A transposition cipher changes the **order** of characters (permutation), not the characters themselves.
Goal is mainly **diffusion**.

Keyless Transposition

- Write into a table and read out by a different direction (row/column variations)

Keyed Transposition

- Uses a key to permute characters within groups
- Extra filler characters may be added to complete a group and later discarded

Combined Approach

- Write row-by-row → reorder columns → read column-by-column

Making a Good Encryption Algorithm

Shannon's Characteristics of a "Good" Cipher

A good cipher should:

1. Match the security need with reasonable effort
2. Avoid unnecessary complexity/restrictions (keys + plaintext should be practical)
3. Be as simple as possible to implement correctly
4. Not allow one small error to corrupt the entire message
5. Not expand ciphertext unnecessarily (large expansion helps attackers and wastes resources)

Properties of "Trustworthy" Commercial Encryption

A trustworthy encryption system is:

- Based on **sound mathematics**
- Reviewed by **competent experts**
- Tested over time and proven reliable in practice

Cryptography Types (High-Level)

Cryptographic algorithms are broadly divided into:

- **Symmetric key cryptography**
- **Asymmetric cryptography**

Playfair Cipher

The **Playfair Cipher** is a classical **digraph substitution cipher**, meaning it encrypts plaintext **two letters at a time** (pairs). It uses a **5×5 key square** generated from a keyword.

Key Square Generation (5×5)

Key Square Rules

- Create a **5×5 grid** of letters (25 cells).
- Since English has 26 letters, one letter is omitted — usually **J**.
- If plaintext contains **J**, replace it with **I**.
- Fill the square using:
    1. **Unique letters of the key** (in order)
    2. Then the **remaining alphabet letters** (excluding J), in order

Example Key: MONARCHY

Remove duplicates → **M O N A R C H Y** then continue with remaining letters (excluding J).

Key Square:

M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z
(I and J share one cell: I/J)

Preparing the Plaintext (Digraph Formation)

Step: Split into pairs (digraphs)

- Break the plaintext into **pairs of 2 letters**.
- If the last letter is alone (odd length), add a filler letter (commonly **Z**).

Important Rule: No pair can have the same letters

If a pair contains the **same letter twice**, insert a fake letter (commonly **X**) between them and re-pair.

Examples

Example A: instruments

Plaintext: **instruments** (11 letters → odd)

Pairs:

- **in st ru me nt sz**
  Explanation: last **s** is alone → add **z** → **sz**

Example B: hello

Pairs:

- **he lx lo**
  Explanation: ll cannot be a pair → insert **x** → helxlo

Example C: helloe

Pairs:

- **he lx lo ez**
  Explanation: ll breaks → insert **x**, and last e alone → add **z**

Encryption Rules (Playfair)

Encrypt **each digraph** using the key square:

Rule 1: Same Column

If both letters are in the **same column**:

- Replace each letter with the **letter below it**
- If at the bottom, wrap to the top

Example: **me**

- m → c
- e → l
  Cipher: **cl**

Rule 2: Same Row

If both letters are in the **same row**:

- Replace each letter with the **letter to the right**
- If at the rightmost, wrap to the leftmost

Example: **st**

- s → t

- t → l
  Cipher: **tl**

Rule 3: Rectangle Rule (Different row and column)

If letters are in **different rows and columns**:

- Form a rectangle using the two letters
- Replace each letter with the letter in the **same row** but in the **column of the other letter** (horizontal opposite corners)

Example: **nt**

- n → r
- t → q
  Cipher: **rq**

Full Encryption Example

Key: MONARCHY

Plaintext: instruments

Digraphs:
in | st | ru | me | nt | sz

Now encrypt each pair:

- in → **ga**
- st → **tl**
- ru → **mz**
- me → **cl**
- nt → **rq**
- sz → **tx**

Ciphertext: gatlmzclrqtx

Decryption (Reverse Process)

Decryption uses the **same key square**, and the ciphertext is already an **even number of letters** (always digraphs).

Rule 1: Same Column

- Move **up** instead of down
- Wrap bottom if needed

Rule 2: Same Row

- Move **left** instead of right
- Wrap rightmost if needed

Rule 3: Rectangle Rule

- Same rectangle rule works
- Take the **horizontal opposite corners** again (it naturally reverses)

Example Decryption

Ciphertext: gatlmzclrqtx

Pairs:

- ga → in
- tl → st
- mz → ru
- cl → me
- rq → nt
- tx → sz

Transposition Cipher

A **transposition cipher** is a classical encryption technique where the **positions of plaintext characters are rearranged** according to a fixed rule. The characters themselves are not replaced—only their **order changes**, so the ciphertext becomes a **permutation** of the plaintext.
Mathematically, encryption applies a **bijective function** on positions, and decryption uses the **inverse function** to restore the original order.

Key Idea (What it does)

- **Substitution cipher** changes characters → aims at *confusion*
- **Transposition cipher** changes positions → aims at *diffusion* (spreads patterns)

Since letter frequencies remain the same, basic transposition can often be detected using frequency analysis.

Types / Implementations of Transposition Cipher

Rail Fence Cipher

The **Rail Fence cipher** writes plaintext in a zig-zag pattern across multiple “rails” and then reads it row by row.

How it works (concept)

- Choose number of rails (e.g., 3)

- Write the message diagonally down and up repeatedly
- Read each rail row-wise to produce ciphertext

Strength: simple and fast
Weakness: can be broken if attacker guesses the rail count and reconstructs the zig-zag.

Route Cipher

A **route cipher** writes plaintext in a grid of fixed dimensions, then reads it out using a route defined by a key (like spiral, zig-zag, clockwise, etc.).

Key point

The number of possible routes can be very large, making it stronger than rail fence. However, poor route choices may leave plaintext chunks visible or reversed, helping cryptanalysts.

Historical note: A variation used in the American Civil War transposed **whole words** instead of letters (Union Route Cipher).

Columnar Transposition

In **columnar transposition**, plaintext is written in rows under a keyword, and the ciphertext is read column by column based on the **alphabetical order** of the keyword letters.

Steps

1. Choose a keyword (e.g., ZEBRAS)
2. Write the message row-wise into a table with columns = keyword length
3. Reorder columns according to the keyword's alphabetical ranking
4. Read down columns in that order

Regular vs Irregular

- **Regular columnar transposition:** fill empty spaces with null letters (padding)
- **Irregular columnar transposition:** leave spaces blank (no padding)

Stronger than simple rail fence
Still vulnerable if attacker guesses column length and uses anagramming techniques.

Double Transposition

A **double transposition** applies columnar transposition **twice**.

- It may use the **same key twice**, or **two different keys**
- This significantly increases security compared to single transposition

Weak point: If many messages of the same length are encrypted using the same keys, attackers can solve them together (simultaneous anagramming).

Historical use: used in WWI, WWII resistance groups, intelligence operations; considered one of the most complex field-operable manual ciphers before modern systems.

Myszkowski Transposition

This is a variant of columnar transposition that requires a keyword with **repeated letters**.

Key difference

- Normal columnar: repeated letters get different ranks
- Myszkowski: repeated letters get the **same rank**

This changes how columns are read:

- Columns with unique numbers are read top-to-bottom
- Columns with repeated numbers are read left-to-right across those columns

This introduces irregularity and makes pattern recovery harder.

Disrupted Transposition

In **disrupted transposition**, some positions in the grid are intentionally left unused (blanked out).
This breaks neat patterns and increases difficulty for cryptanalysis.

Grilles

A **grille cipher** uses a physical mask (a sheet with cut-out holes) placed over a grid.

- The sender writes text only through the holes
- Then moves/rotates the grille and continues
- The receiver must have the same grille to read correctly

Produces highly irregular transpositions
Requires physically protecting the grille (key).

Detection and Cryptanalysis

Because transposition keeps letter frequencies unchanged, it can be detected by:

- **Frequency counting** (ciphertext frequency looks like normal English/plaintext)

Common attack method:

- **Anagramming**: shifting blocks around to find readable word fragments
- Guessing **column lengths**, reconstructing grids, and trying column orders

Also, keys "close" to the correct key may reveal partially readable text mixed with nonsense, helping attackers refine guesses.

Combinations

Transposition is often combined with other methods, especially substitution.

Example: substitution + columnar transposition

- Substitution hides frequencies
- Transposition hides readable word chunks
  This combination removes the biggest weaknesses of both methods, but becomes more laborious and error-prone.

Fractionation

Fractionation breaks each plaintext symbol into multiple ciphertext symbols first, then transposes them. This spreads parts of letters across the message → strong diffusion.

Examples of fractionation:

- Polybius square coordinates
- Straddling checkerboard
- Converting text to Morse code then transposing

Ciphers that combine fractionation + transposition include:

- Bifid, Trifid, ADFGVX, VIC

This approach is conceptually similar to modern block cipher design where diffusion is achieved by splitting and mixing data repeatedly.

Hashing

Hashing is a data security technique used to convert data values into alternate, unique identifiers called **hashes**. It is widely used for **quick access** and **security** because hashing is a **one-way process**—meaning you cannot easily reverse a hash to get the original data.

Hashing helps in security because it prevents:

- Direct access to original sensitive data
- Easy tampering (because even small changes produce very different hashes)

What is a Hash Function?

A **hash function** is an algorithm that converts **any amount of input data** into a **fixed-length output** (called a hash value / digest).

Key property of a good hash function

A good hash function has the **avalanche effect**:

- A **tiny change** in input (even one letter) produces a **dramatically different** output hash.

This property is extremely important in security because it becomes nearly impossible to:

- Predict the hash
- Modify data without detection
- Derive the original input from the hash

Hashing in Digital Signatures (Use Case)

Hashing is used in **digital signatures** to ensure:

- Authenticity
- Integrity (no tampering)

How it works (simple flow)

1. Create a **hash** of the document.
2. Encrypt that hash using the sender's **private key** → this becomes the **digital signature**.
3. Receiver uses the sender's **public key** to decrypt the signature.
4. Receiver recomputes the hash of the received document.

5. If both hashes match → document is authentic and unchanged.

Matching hashes = data not modified

Hashing vs Encryption (Difference)

Encryption

- Converts plaintext into **ciphertext**
- **Reversible** using a key (decrypt to get original data)

Hashing

- Converts plaintext into a **hash**
- **Not reversible** (you cannot convert hash back to original data)

One-line difference:
Encryption is **two-way**, hashing is **one-way**.

Limitations of Hashing

Even though hashing is useful, it has a few limitations:

a) Hash Collision

A **collision** happens when:

- Two different inputs produce the **same hash output**

Good hash functions make collisions extremely unlikely, but not impossible.

b) No Original Data Recovery

Hashing is one-way, so:

- You **cannot retrieve** the original message from the hash

This is good for password protection, but not suitable when you need the original data back.

Historical Hash Algorithms

Early hashing algorithms include:

- **MD2, MD4, MD5**
  These generate **128-bit** message digests and were developed by **Ron Rivest**, commonly used in early digital signature systems.

The **SHA family** (Secure Hash Algorithm) was developed through collaboration of:

- **NIST** and **NSA**

SHA versions include:

- **SHA-1**
- **SHA-256**
- **SHA-384**
- **SHA-512**

They differ mainly by **hash length**, and longer hashes generally provide stronger security.

Hashing Algorithm

Hashing is also used in data structures (hash tables) to map keys to an **index/address**.

Hashing Numeric Keys

For numeric keys:

$$\text{Address} = \text{Key mod } n$$

Where **n** = number of available addresses (table size).

Example idea:

- If n = 10 and key = 57
  Address = 57 mod 10 = 7

Hashing Alphanumeric Keys

For alphanumeric keys:

1. Convert characters to ASCII values
2. Take the sum of ASCII values
3. Apply mod n

$$\text{Address} = (\sum ASCII) \text{mod } n$$

Folding Method

The **folding method** divides a key into equal parts and adds them together.

Example (telephone number):

- 01452 8345654
  Split and add:
- 01 + 45 + 28 + 34 + 56 + 54 = **218**

Then (if needed) use:

- 218 mod n (based on table size)

Collision in Hashing (Hash Table Collision)

What is a Collision?

A collision occurs when a hash function maps two or more different keys to the **same index** in a hash table.
Since one slot usually holds one value, collisions must be handled using collision resolution techniques.

Example idea

If **Mia** and **Sue** both map to index **4**, then collision happens.

Collision Resolution: Linear Probing

Linear probing solves collisions by searching for the next empty slot.

How it works

If the index is occupied:

- check index + 1
- then index + 2
- continue until an empty slot is found

Example explanation (your case)

- Mia stored at index 4
- Sue also maps to index 4, but it's occupied
- So Sue is stored at the **next available slot**, e.g., index 6 (after checking 5 etc.)